

Elite Events

Updated Software Specification

Event reservation, ticketing, QR check-in, multi-tenant administration, and SaaS subscription design

Document status Updated baseline and planned SaaS extension	Application stack Angular 18 frontend Express + TypeScript API PostgreSQL
Version date 18 August 2026	Audience Product, engineering, operations, and stakeholders

This revision supersedes the original single-admin specification. It documents the implemented client-tenant architecture and records the proposed subscription model as planned scope until payment integration is delivered.

Contents

1. Purpose, scope, and current status
2. Product architecture and tenant model
3. Roles and access control
4. Venue, events, and seating model
5. Reservations, tickets, and QR check-in
6. Authentication and account management
7. Implemented interface capabilities
8. SaaS subscription model - planned implementation
9. Billing, CamPay payment, and lifecycle design
10. Data model additions and API surface
11. Security, quality, and operational requirements
12. Delivery phases and acceptance criteria

Reading convention

Implemented describes behavior present in the application. Planned describes approved design direction for the SaaS subscription change and is not yet production functionality.

1. Purpose, scope, and current status

Elite Events is a web application for configuring venues, scheduling events, allocating seats, issuing QR-code tickets, and validating entry at the gate. The system separates permanent venue data from event-specific seating and occupancy.

This revision captures the application after its transition from a single administrative account to a multi-tenant product. It also defines the next planned transition: subscription-based SaaS access for Client Admin organizations.

Implemented baseline

- Angular 18 standalone frontend and Express + TypeScript backend using PostgreSQL with raw SQL migrations.
- Three operational roles: Super Admin, Client Admin, and Gate Staff.
- Tenant isolation for client-owned buildings, events, invitees, and Gate Staff accounts.
- Multi-room events with conflict validation for every selected room.
- Event-scoped tables and chairs, single and couple reservations, PDF tickets, QR generation, and gate scanning.
- JWT authentication, OTP verification for administrative roles, protected routes, and login rate limiting.

Planned SaaS extension

Subscriptions, billing, CamPay collections, quotas, plan-aware UI, and self-service client onboarding are planned. They must be implemented as a backend-enforced entitlement system; frontend controls are only a convenience layer.

2. Product architecture and tenant model

A Client Admin is the root of an organization, also called a tenant. All resources created by that Client Admin are logically owned by that tenant. Gate Staff accounts created by a Client Admin inherit the same tenant ownership.

Layer	Responsibility	Current behavior
Frontend	Angular application	Role-aware navigation, protected routes, dashboard, venues, events, seating, ticket preview, scanner, staff and client screens.
API	Express service layer	JWT authentication, role checks, validation with Zod, tenant-scoped SQL queries, ticket and QR operations.
Database	PostgreSQL	Foreign keys, unique constraints, migrations, client ownership columns, event schedule indexes, and anti-double-booking constraint.
SaaS module - planned	Entitlements and payments	Subscription status, quotas, billing transactions, CamPay adapter, webhook processing, and reconciliation.

Tenant ownership rules

- Buildings and events created by a Client Admin receive that Client Admin ID.
- Rooms are tenant-scoped through their parent building.
- Invitees are tenant-owned. Gate Staff accounts are tenant-owned and may only be assigned to that tenant's events.
- Client Admin list, read, update, and delete operations are filtered to their own tenant. Super Admin may administer all tenants.
- Subscription state will be attached to the Client Admin tenant, not to a Gate Staff account or a specific building.

3. Roles and access control

Role	Scope	Key capabilities
Super Admin	Platform-wide	Creates and manages Client Admin accounts; views platform data; can manage venue and event resources; bypasses tenant quotas for platform operations.
Client Admin	Own tenant only	Manages own buildings, rooms, events, event seating, invitees, tickets, reservations, and Gate Staff. Will manage the tenant subscription and payments.
Gate Staff	Assigned events only	Logs in with password; scans QR codes and validates ticket entry for assigned events. Cannot manage tenant resources or billing.

Authorization requirements

- Every protected API request carries a JWT. The API validates it before applying role checks.
- Tenant filtering is performed in backend queries and services. Client-side route guards do not replace server authorization.
- A Client Admin must never access another Client Admin's resources by modifying an identifier in a URL or request body.
- Planned plan limits must be enforced server-side before creating a building or event, including concurrent requests.

Important implementation note

The original generic Admin role is now represented by two roles: Super Admin for platform administration and Client Admin for tenant administration. Existing documentation and UI copy should use these names.

4. Venue, events, and seating model

The implemented model distinguishes a reusable physical venue from an event-specific seating plan.

Physical venue

- A Building contains Rooms. Each Room has a room number, floor number, and optional capacity.
- A Room may be used by many events over time but cannot be selected for overlapping event windows.
- Buildings and Rooms are persistent tenant resources.

Events and seating

- An Event has a name, start time, end time, primary room, and one or more selected rooms.
- Every selected room is checked for scheduling conflicts. Events whose end time plus the configured 30-minute grace period has passed are hidden from active operational lists.
- Tables and chairs belong to an Event, not to the permanent Room layout. This allows different seating layouts for different events in the same room.
- Each event table is linked to a selected room. Chair numbers are unique within their table.
- An event can be edited when ownership and schedule validation pass. A room assignment cannot be changed after tickets have been issued.

Core schedule constraint

For each room used by an event, the system rejects an overlapping interval: existing start is before proposed end and existing end is after proposed start. This applies to the primary room and every room in the multi-room selection.

5. Reservations, tickets, and QR check-in

Reservations represent event-scoped occupancy. A chair is never globally reserved; it is reserved only for a particular event.

Entity	Implemented behavior
Invitee	Stores name and optional email/phone, associated with the Client Admin tenant.
Reservation	Links event, event table, chair, invitee, room, status, and reservation type. The database prevents more than one ACTIVE reservation for the same event and chair.
Single reservation	One invitee occupies one chair and receives a ticket.
Couple reservation	One invitee occupies two valid, free chairs on the same table. The reservations share a couple group ID so cancellation is coordinated.
Ticket	Created for the primary reservation with a unique non-guessable QR token. Status is ISSUED, CHECKED_IN, or CANCELLED.

Check-in rules

- Gate Staff may scan only events to which they are assigned.
- The QR token is looked up server-side; it does not expose the invitee's personal data or seat details.
- A cancelled ticket is rejected. A checked-in ticket is rejected on a repeat scan.
- Cancelling a couple reservation cancels the paired reservations and related tickets in the same couple group.
- After an event is finished, seating modifications are blocked.

6. Authentication and account management

Account type	Login flow	Account management
Super Admin	Environment-configured username/password followed by emailed 8-character OTP.	Platform credentials are configured in the deployment environment.
Client Admin	Database username/password followed by emailed 8-character OTP.	Created, updated, deactivated, reactivated, or deleted by Super Admin in the implemented baseline.
Gate Staff	Database username/password; no OTP.	Created and managed by Super Admin or owning Client Admin; can be assigned to multiple events.

Security controls

- OTP has limited lifetime and attempt count. Authentication routes are rate limited.
- JWTs expire and are attached by the frontend HTTP interceptor to protected requests.
- Client deactivation blocks Client Admin login. Gate Staff deactivation blocks Gate Staff login.
- Passwords are stored as bcrypt hashes; credentials and mail secrets must remain outside source control.

7. Implemented interface capabilities

- Dashboard: current rooms, buildings, and live/upcoming event visibility.
- Venue management: create/list/delete buildings and rooms within tenant scope.
- Event management: create multi-room events, define event-specific tables and chairs, edit scheduling subject to issued-ticket constraints.
- Seating and guest list: inspect occupancy, create single or couple reservations, issue tickets, and manage cancellations.
- Ticket preview and scanner: display tickets and scan QR tokens for assigned gate operations.
- Super Admin client management: create, edit, deactivate, reactivate, and permanently delete Client Admin accounts.

8. SaaS subscription model - planned implementation

Subscriptions are a tenant-level commercial entitlement. They do not alter tenant ownership or authorization; they decide which new resources the tenant can create and which paid features it can use.

Plan	Buildings	Events	Commercial intent
Free	1 active building	2 event creations per monthly billing period	Entry plan for evaluation and low-volume use.
Go	3 active buildings	4 event creations per monthly billing period	Paid plan for growing event organizers.
Pro	Unlimited	Unlimited	Full access and future premium functionality.

Plan assumptions requiring product confirmation

- Go and Pro monthly prices in XAF are intentionally not fixed in this document and must be approved before checkout is released.
- An event quota is measured by event creation during the current billing period. Deleting an event does not restore an event quota, preventing quota abuse.
- A building quota counts active buildings. Deleting a building restores capacity if the deletion is valid under existing data constraints.
- All current operational functions remain available to Free, Go, and Pro unless a future feature is explicitly designated as paid-only. Pro means no resource quota, not exemption from security or data-integrity rules.
- On downgrade or expiry, data remains intact. New creation is blocked only when the tenant is over the effective limit.

Implementation principle

The frontend may show a disabled action or upgrade prompt, but the backend must calculate and enforce the allowance in the same transaction that creates the building or event.

9. Billing, CamPay payment, and lifecycle design

CamPay will be used for Mobile Money collection. Payment credentials and provider calls remain on the Express backend; the Angular application never receives a CamPay secret.

Checkout and activation

- Client Admin selects Go or Pro and supplies an eligible Mobile Money number.
- The backend creates an internal pending payment record and initiates a CamPay collection request.
- The payer confirms the payment on the phone. The frontend can poll its internal payment endpoint for status.
- The paid subscription activates only after the backend receives and verifies a successful CamPay confirmation or reconciles a confirmed provider status. A frontend redirect alone is not proof of payment.

Subscription lifecycle

State	Meaning	Expected behavior
FREE	No paid period	Free limits apply.
PENDING_PAYMENT	Collection initiated	No paid entitlement until confirmation.
ACTIVE	Paid period confirmed	Go or Pro entitlements apply until period end.
PAST_DUE	Paid period ended; optional grace period	Notify Client Admin; preserve data; policy controls temporary access.
CANCEL_AT_PERIOD_END	Renewal cancelled	Paid benefits continue through current period end.
EXPIRED	No active paid entitlement	Revert to Free limits without deleting data.

Renewal approach

Initial release should use application-managed monthly renewal: create and collect a new payment for each billing period, with notices before expiry. Automatic recurring Mobile Money debit must not be introduced until CamPay confirms the appropriate API, customer consent flow, and production requirements.

10. Data model additions and API surface

The current schema is tenant-aware. The following additions are required for SaaS billing.

Data object	Key fields and purpose
subscriptions	client_id, plan_code, status, current_period_start, current_period_end, cancel_at_period_end, price snapshot. One current subscription per Client Admin tenant.
payment_transactions	client_id, intended_plan, amount_xaf, provider reference, payment phone/operator, status, initiated/confirmed/failed timestamps, idempotency key.
payment_webhook_events	provider event ID, received timestamp, raw payload, processing status. Supports audit and idempotent callback handling.
usage_ledger	client_id, billing period, usage type, related resource ID, created timestamp. Immutable event-creation records support monthly quota enforcement.
plan definitions	Code-owned configuration initially: limits, display name, price, interval, and enabled features. May move to managed storage later.

Planned API endpoints

Endpoint	Purpose
GET /plans	Return published plans and public pricing.
GET /billing/subscription	Return tenant plan, status, period, and effective entitlements.
GET /billing/usage	Return building count and current-period event usage.
POST /billing/checkout	Create pending payment and initiate CamPay collection.
GET /billing/payments/:id	Return internal payment status for polling.
POST /billing/cancel	Request cancellation at the end of the paid period.
POST /webhooks/campay	Receive and idempotently process verified CamPay notifications.

11. Security, quality, and operational requirements

- CamPay keys, webhook verification material, SMTP credentials, JWT secret, and environment-admin credentials must be stored in deployment secrets, never in source control or the frontend.
- Use HTTPS for production API and webhook traffic. Validate and authenticate incoming provider callbacks according to CamPay production documentation.
- Webhook processing must be idempotent. Replayed or duplicated notifications must not create duplicate paid periods or change a completed payment incorrectly.
- Record payment state changes with audit-safe metadata. Do not log credentials or unnecessarily expose customer payment information.
- Run scheduled reconciliation for pending payments and expiring subscriptions. Surface unresolved payments to Super Admin operations.
- Test quota enforcement under concurrent requests, not only through UI interactions.

12. Delivery phases and acceptance criteria

Phase	Deliverable	Acceptance criteria
1. Billing foundation	Migrations, plan definitions, subscription and usage services.	Existing clients are safely backfilled; Free limits calculate correctly; no data is deleted.
2. Entitlements	Transactional building/event quota checks and billing read APIs.	Limits cannot be bypassed by direct API calls or concurrent creation requests.
3. Payments	CamPay adapter, checkout, callback verification, reconciliation.	Sandbox successful, failed, pending, duplicate, and invalid callback paths are tested.
4. User experience	Pricing/billing screen, usage indicators, upgrade path, client admin billing controls.	Client Admin can understand status, quota, and payment outcome without contacting support.
5. Production rollout	Live credentials, monitoring, support runbook, controlled release.	Real payment validation completed; stale pending payments and subscription expiry are observable.

Final scope statement

Elite Events is now specified as a multi-tenant event operations platform. The next implementation milestone is a subscription layer that preserves tenant isolation and operational safety while enforcing Free, Go, and Pro commercial entitlements through verified CamPay payments.